



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

LABORATORIO DE MICROCOMPUTADORAS

**Práctica 03 Programación en ensamblador,
direccionamiento indirecto**

Grupo de laboratorio: 02

Profesor: Ing. Moisés Meléndez Reyes

Brigada: 04

Integrantes:

- Hernandez Acosta Mauricio Gabriel 321121051
- Pérez Olvera Alexis Abraham 321150503
- Sierra Garcia Mariana 320301238



Fecha de entrega: Domingo 27 de septiembre del 2026

Semestre: 2027-1

Índice

1. Objetivo	2
2. Actividades	2
2.1. Ejercicio 1	2
2.1.1. Pseudo Código	2
2.1.2. Desarrollo	2
2.1.3. Análisis de resultados	2
2.2. Ejercicio 2	3
2.2.1. Pseudo código	3
2.2.2. Desarrollo	3
2.2.3. Análisis de Resultados	4
2.3. Ejercicio 3	5
2.3.1. Pseudo código	5
2.3.2. Desarrollo	5
2.3.3. Análisis de Resultados	5
2.4. Ejercicio 4	6
2.4.1. Pseudo código	6
2.4.2. Desarrollo	6
2.4.3. Análisis de Resultados	7
2.5. Ejercicio 5	8
2.5.1. Pseudo código	8
2.5.2. Desarrollo	8
2.5.3. Análisis de Resultados	9
2.6. Ejercicio 6	10
2.6.1. Pseudo código	10
2.6.2. Desarrollo	10
2.6.3. Análisis de Resultados	11
2.7. Ejercicio 7	12
2.7.1. Pseudo código	12
2.7.2. Desarrollo	12
2.7.3. Análisis de Resultados	13
2.8. Ejercicio 8	14
2.8.1. Pseudo código	14
2.8.2. Desarrollo	14
2.8.3. Análisis de Resultados	15
2.9. Ejercicio 9	16
2.9.1. Pseudo código	16
2.9.2. Desarrollo	16
2.9.3. Análisis de Resultados	17
2.10. Ejercicio 10	18
2.10.1. Pseudo código	18
2.10.2. Desarrollo	19
2.10.3. Análisis de Resultados	20
3. Conclusiones	21
3.1. Hernandez Acosta Mauricio Gabriel	21
3.2. Perez Olvera Alexis Abraham	21
3.3. Sierra Garcia Mariana	21

1. Objetivo

Conocer los recursos de la plataforma con el microcontrolador RP2040 contenida en la plataforma Raspberry Pi Pico; desarrollar algoritmos mediante la programación en Micropython utilizando el IDE Thonny para generar salidas a través de las terminales GPIO.

2. Actividades

2.1. Ejercicio 1

Usando el IDE Thonny; escribir el siguiente código y comentar cada instrucción:

```
1 import machine
2 import utime
3 LED = machine.Pin(0, machine.Pin.OUT)
4 while True:
5     LED.value(1)
6     utime.sleep_ms(500)
7     LED.value(0)
8     utime.sleep_ms(500)
```

2.1.1. Pseudo Código

Algoritmo 1: Encendido y apagado periódico de un LED

Configurar GPIO0 como salida;

Mientras verdadero hacer;

 Encender LED;

 Esperar 500 ms;

 Apagar LED;

 Esperar 500 ms;

2.1.2. Desarrollo

```
1 import machine
2 import utime
3
4 # LED de Salida
5 LED = machine.Pin(0, machine.Pin.OUT)
6
7 while True:
8     LED.value(1) # LED prendido
9     utime.sleep_ms(500) # 0.5 segundos de espera
10    LED.value(0) # LED apagado
11    utime.sleep_ms(500) # 0.5 segundos de espera
```

2.1.3. Análisis de resultados

Cuando programamos la tarjeta con el código podemos observar el primer LED color verde prenderse y apagarse en intervalos de medio segundo por lo que el comportamiento es el adecuado al descrito en el código.

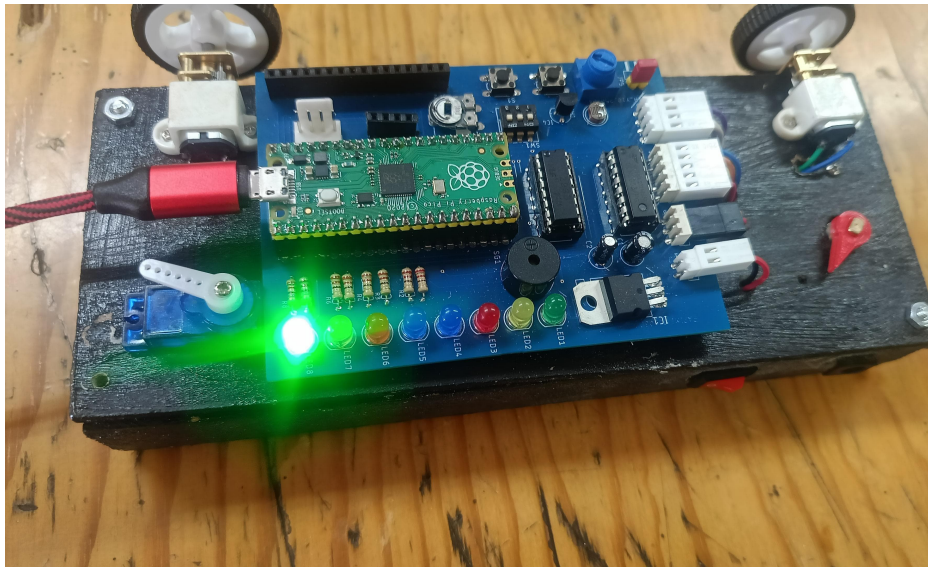


Figura 1: Tarjeta programada con el Ejercicio 1

2.2. Ejercicio 2

Realizar las modificaciones necesarias para que los tiempos sean más pequeños y más grandes que el código anterior.

2.2.1. Pseudo código

Algoritmo 2: Encendido y apagado del LED con tiempos modificados

Configurar GPIO0 como salida;

Mientras verdadero hacer;

 Encender LED;

 Esperar 1000 ms;

 Apagar LED;

 Esperar 300 ms;

2.2.2. Desarrollo

```
1 import machine
2 import utime
3
4 # LED de Salida
5 LED = machine.Pin(0,machine.Pin.OUT)
6
7 while True:
8     LED.value(1) # LED prendido
9     utime.sleep_ms(1000) # 1 segundo de espera
10    LED.value(0) # LED apagado
11    utime.sleep_ms(300) # 0.3 segundos de espera
```

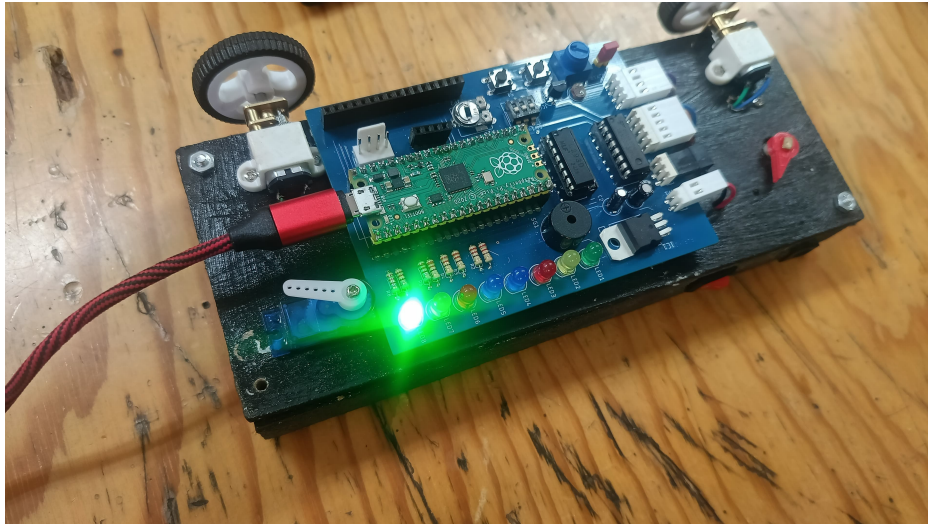


Figura 2: Tarjeta programada con el Ejercicio 2

2.2.3. Análisis de Resultados

De manera similar al ejercicio 1, cuando programamos la tarjeta el LED verde prende y apaga pero esta vez permanece prendido durante 1 segundo completo, comportándose de manera adecuado de acuerdo al código escrito.

2.3. Ejercicio 3

Realizar las modificaciones requeridas para que el efecto se muestre en otro GPIO (podrá usar GPIO1 al GPIO 7); usar retardos a conveniencia.

2.3.1. Pseudo código

Algoritmo 3: Control de LED en otro GPIO

Configurar GPIO2 como salida;

Mientras verdadero hacer;

 Encender LED;

 Esperar 1200 ms;

 Apagar LED;

 Esperar 700 ms;

2.3.2. Desarrollo

```
1 import machine
2 import utime
3
4 # LED de salida
5 LED = machine.Pin(2,machine.Pin.OUT)
6
7 while True:
8     LED.value(1) # LED prendido
9     utime.sleep_ms(1200) # 1.2 segundos de espera
10    LED.value(0) # LED apagado
11    utime.sleep_ms(700) # 0.7 segundos de espera
```

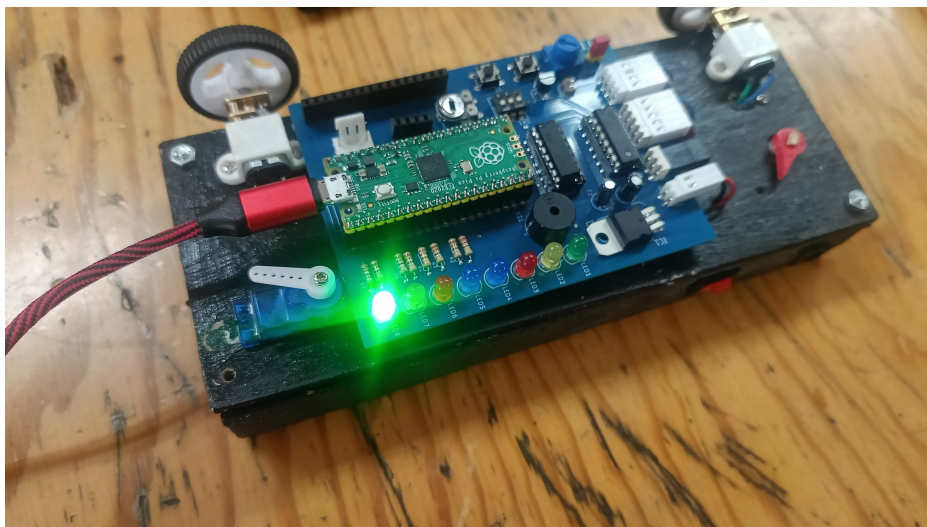


Figura 3: Tarjeta programada con el Ejercicio 3

2.3.3. Análisis de Resultados

Una vez más, al programar la tarjeta podemos ver el LED verde prender y apagar con intervalos de 1.2 segundos y 0.7 segundo de acuerdo al código escrito.

2.4. Ejercicio 4

Realizar un programa que genere las siguientes salidas:

GPIO2	GPIO3	@85 ms entre estados
1	0	
0	1	

2.4.1. Pseudo código

Algoritmo 4: Encendido alternado de dos LEDs

Configurar GPIO2 como salida;

Configurar GPIO3 como salida;

Mientras verdadero hacer;

 Encender LED en GPIO2;

 Apagar LED en GPIO3;

 Esperar 85 ms;

 Apagar LED en GPIO2;

 Encender LED en GPIO3;

 Esperar 85 ms;

2.4.2. Desarrollo

```
1 import machine
2 import utime
3
4 # LED's de salida
5 LEDg2 = machine.Pin(2,machine.Pin.OUT)
6 LEDg3 = machine.Pin(3,machine.Pin.OUT)
7
8 while True:
9     LEDg2.value(1) # LED rojo prendido
10    LEDg3.value(0) # LED azul apagado
11    utime.sleep_ms(85) # 0.085 segundos de espera
12    LEDg2.value(0) # LED rojo apagado
13    LEDg3.value(1) # LED azul prendido
14    utime.sleep_ms(85) # 0.085 segundos de espera
```

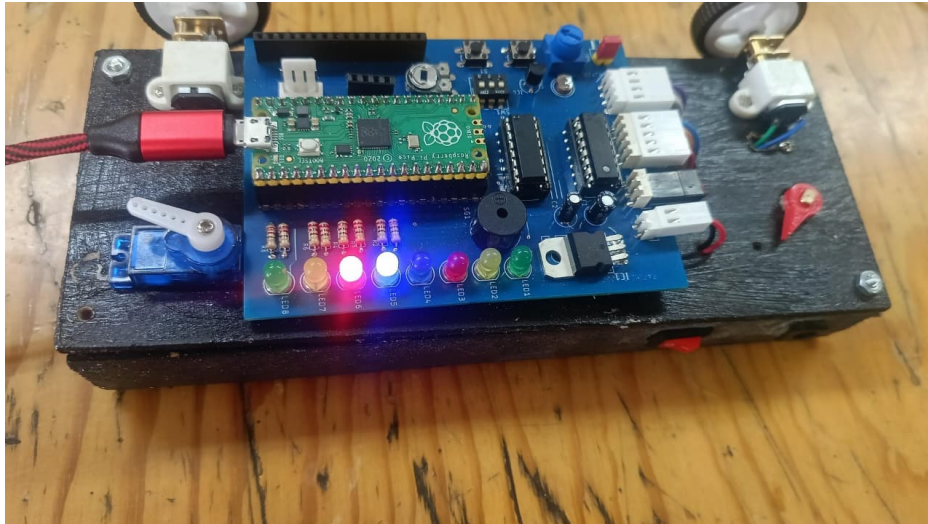


Figura 4: Tarjeta programada con el Ejercicio 4

2.4.3. Análisis de Resultados

Al programar la tarjeta observamos como el LED rojo y azul cambian estados de prendido a apagado intercaladamente siguiendo el comportamiento descrito en el código de manera correcta.

2.5. Ejercicio 5

Realizar un programa que prenda y apague las GPIO0 al GPIO7; el circuito corresponde en su totalidad al mostrado en la figura 3.3, usar los retardos a elección

GPIO0	GPIO1	GPIO2	GPIO3	GPIO4	GPIO5	GPIO6	GPIO7
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1

2.5.1. Pseudo código

Algoritmo 5: Encendido y apagado simultáneo de GPIO0 a GPIO7

Definir lista de GPIO del 0 al 7;

Mientras verdadero hacer;

 Para cada LED en la lista hacer;

 Encender LED;

 Esperar 800 ms;

 Para cada LED en la lista hacer;

 Apagar LED;

 Esperar 800 ms;

2.5.2. Desarrollo

```

1 import machine
2 import utime
3
4 # LED's de salida
5 numero_pin = [0, 1, 2, 3, 4, 5, 6, 7]
6 leds = [machine.Pin(p, machine.Pin.OUT) for p in numero_pin]
7
8 while True:
9     # Prender todos los LED's
10    for led in leds:
11        led.value(1)
12    utime.sleep_ms(800) # 0.8 segundos de espera
13
14    # Apagadar todos los LED's
15    for led in leds:
16        led.value(0)
17    utime.sleep_ms(800) # 0.8 segundos de espera

```

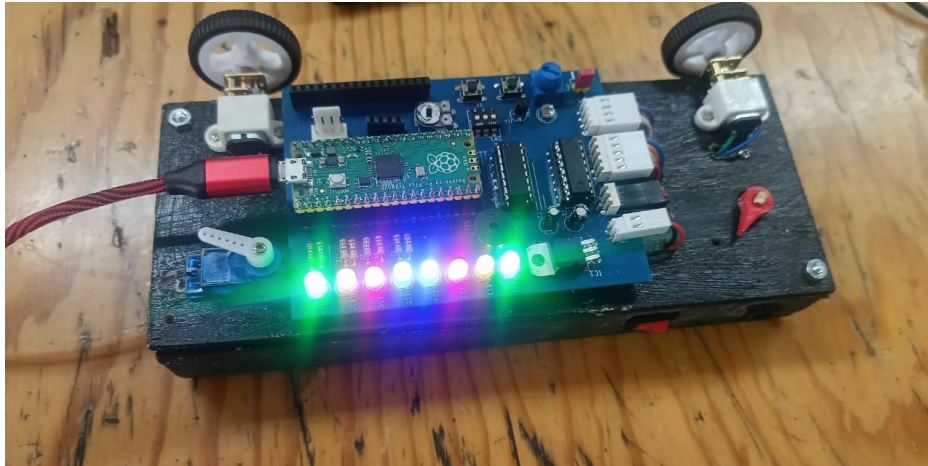


Figura 5: Tarjeta programada con el Ejercicio 5

2.5.3. Análisis de Resultados

Al ejecutar el programa en la tarjeta podemos observar que los 8 LED's prenden simultáneamente y también se apagan simultáneamente de manera acorde al código.

2.6. Ejercicio 6

Repetir la acción anterior 10 veces; mantener apagado 2 segundos y repetir acción.

GPI00	GPI01	GPI02	GPI03	GPI04	GPI05	GPI06	GPI07	
0	0	0	0	0	0	0	0	10 repeticiones (200 ms)
1	1	1	1	1	1	1	1	
0	0	0	0	0	0	0	0	
1	1	1	1	1	1	1	1	
0	0	0	0	0	0	0	0	2 segundos



2.6.1. Pseudo código

Algoritmo 6: Encendido y apagado de LEDs repetido 10 veces

Definir lista de GPIO del 0 al 7;

Mientras verdadero hacer;

 Repetir 10 veces;

 Para cada LED en la lista hacer;

 Encender LED;

 Esperar 200 ms;

 Para cada LED en la lista hacer;

 Apagar LED;

 Esperar 200 ms;

 Esperar 2000 ms;

2.6.2. Desarrollo

```

1 import machine
2 import utime
3
4 # LED's de salida
5 numero_pin = [0, 1, 2, 3, 4, 5, 6, 7]
6 leds = [machine.Pin(p, machine.Pin.OUT) for p in numero_pin]
7
8 # Tiempos de retardo
9 retardo_rep = 200
10 retardo_espera = 2000
11
12 while True:
13     # 10 repeticiones
14     for i in range(10):
15         # Prender todos los LED's
16         for led in leds:
17             led.value(1)
18         utime.sleep_ms(retardo_rep) # 0.2 segundos de espera
19
20         # Apagar todos los LED's
21         for led in leds:
22             led.value(0)
23         utime.sleep_ms(retardo_rep) # 0.2 segundos de espera
24

```

```
25 # Fin de las 10 repeticiones  
26 utime.sleep_ms(retardo_espera) # 2 segundos de espera
```

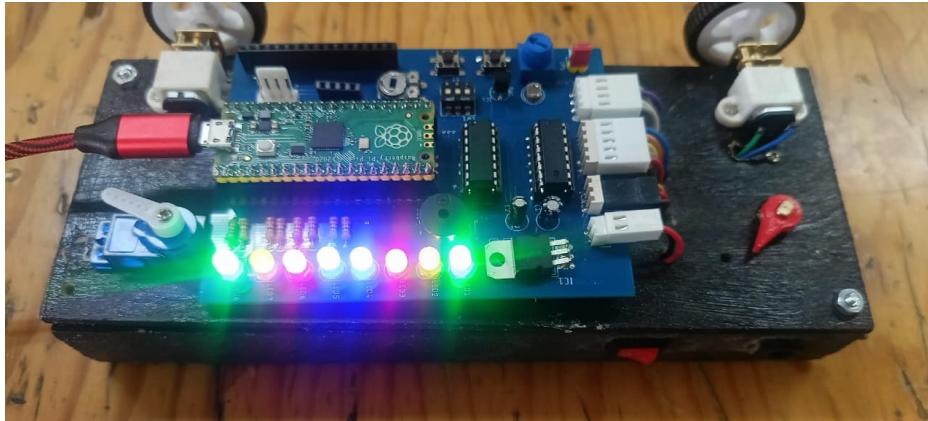


Figura 6: Tarjeta programada con el Ejercicio 6

2.6.3. Análisis de Resultados

Al programar la tarjeta podemos observar 8 LED's prenderse y apagarse simultáneamente un total de 10 veces con intervalos de 0.2 segundos y al finalizar las 10 repeticiones hay un tiempo de espera de 2 segundos antes de comenzar nuevamente con las 10 repeticiones, de forma que el comportamiento descrito por el código se ejecuta correctamente.

2.7. Ejercicio 7

Usando los GPIO0 al GPIO7; realizar un programa que genere la siguiente secuencia; usar retardos de 100ms.

GPIO0	GPIO1	GPIO2	GPIO3	GPIO4	GPIO5	GPIO6	GPIO7
1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1



2.7.1. Pseudo código

Algoritmo 7: Encendido secuencial de LEDs

Definir lista de GPIO del 0 al 7;

Mientras verdadero hacer;

 Para cada LED en la lista hacer;

 Encender LED;

 Esperar 100 ms;

 Apagar LED;

2.7.2. Desarrollo

```

1 import machine
2 import utime
3
4 # LED's de salida
5 numero_pin = [0, 1, 2, 3, 4, 5, 6, 7]
6 leds = [machine.Pin(p, machine.Pin.OUT) for p in numero_pin]
7
8 retardo = 100 # Tiempo de retard
9
10 while True:
11     for led in leds:
12         led.value(1) # LED prendido
13         utime.sleep_ms(retardo) # 0.1 segundos de espera
14         led.value(0) # LED apagado

```



Figura 7: Tarjeta programada con el Ejercicio 7

2.7.3. Análisis de Resultados

Al programar la tarjeta con este código se observa que los 8 LED's se encienden uno tras otro de forma secuencial comenzando desde el GPIO0 hasta el GPIO7. Cada LED permanece encendido durante 100 ms antes de apagarse y continuar con el siguiente LED en la secuencia. Este comportamiento genera un efecto visual de desplazamiento de luz a lo largo de los LEDs, confirmando que el ciclo `for` recorre correctamente la lista de pines y que los retardos se ejecutan de acuerdo con el tiempo establecido en el programa.

2.8. Ejercicio 8

Usando los GPIO0 al GPIO7; realizar un programa que genere la siguiente secuencia; usar retardos de 100ms.

GPIO0	GPIO1	GPIO2	GPIO3	GPIO4	GPIO5	GPIO6	GPIO7
1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0
1	1	1	0	0	0	0	0
1	1	1	1	0	0	0	0
1	1	1	1	1	0	0	0
1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1



2.8.1. Pseudo código

Algoritmo 8: Encendido progresivo de LEDs

Definir lista de GPIO del 0 al 7;

Mientras verdadero hacer;

 Para i desde 0 hasta 7 hacer;

 Para j desde 0 hasta i hacer;

 Encender LED j;

 Esperar 100 ms;

 Apagar todos los LEDs;

2.8.2. Desarrollo

```

1 import machine
2 import utime
3
4 numero_pin = [0, 1, 2, 3, 4, 5, 6, 7]
5 leds = [machine.Pin(p, machine.Pin.OUT) for p in numero_pin]
6
7 retardo = 100
8
9 while True:
10     for i in range(len(leds)):
11         for j in range(i+1):
12             leds[j].value(1)
13             utime.sleep_ms(retardo)
14
15     for led in leds:
16         led.value(0)

```

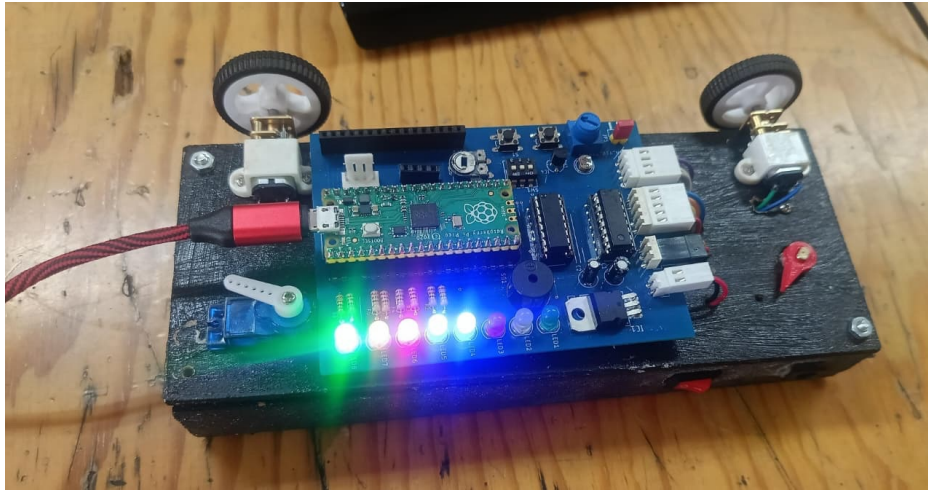


Figura 8: Tarjeta programada con el Ejercicio 8

2.8.3. Análisis de Resultados

Al ejecutar el programa en la tarjeta se observa que los LED's se encienden de forma progresiva. Primero se enciende el primer LED, posteriormente el segundo manteniendo encendido el anterior, y así sucesivamente hasta que los ocho LED's permanecen encendidos al mismo tiempo. Después de completar la secuencia, todos los LED's se apagan y el proceso se repite nuevamente. Este comportamiento confirma que el uso de los ciclos anidados permite controlar el encendido acumulativo de los LEDs conforme a lo especificado en el código.

2.9. Ejercicio 9

Usando los GPIO0 al GPIO7; realizar un programa que genere un contador de 8 bits de forma ascendente; usar retardos de 50ms.

GPIO0	GPIO1	GPIO2	GPIO3	GPIO4	GPIO5	GPIO6	GPIO7
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1



2.9.1. Pseudo código

Algoritmo 9: Contador binario de 8 bits

Definir lista de GPIO del 0 al 7;

Mientras verdadero hacer;

 Para numero desde 0 hasta 255 hacer;

 Para bit desde 0 hasta 7 hacer;

 Obtener valor del bit correspondiente del número;

 Asignar valor al LED correspondiente;

 Esperar 50 ms;

2.9.2. Desarrollo

```

1 import machine
2 import utime
3
4 numero_pin = [0, 1, 2, 3, 4, 5, 6, 7]
5 leds = [machine.Pin(p, machine.Pin.OUT) for p in numero_pin]
6
7 retardo = 50
8
9 while True:
10     for numero in range(256):
11         for bit in range(8):
12             estado_bit = (numero >> bit) & 1
13             leds[7-bit].value(estado_bit)
14             utime.sleep_ms(retardo)

```

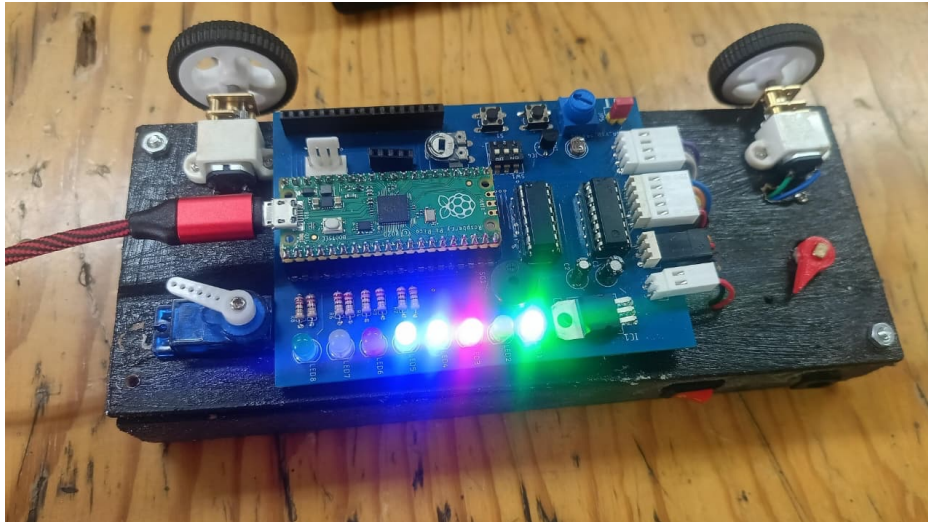


Figura 9: Tarjeta programada con el Ejercicio 9

2.9.3. Análisis de Resultados

Al cargar el programa en la tarjeta se observa que los LED's representan visualmente un contador binario de 8 bits. Los LEDs cambian de estado cada 50 ms mostrando las diferentes combinaciones binarias desde 00000000 hasta 11111111, lo cual corresponde a los valores decimales del 0 al 255. Este comportamiento permite verificar que la operación de desplazamiento de bits y la asignación de cada bit a un LED funcionan correctamente, generando una representación visual del conteo binario ascendente.

2.10. Ejercicio 10

Usando las salidas correspondientes a los colores de los leds; realizar un programa que controle el funcionamiento de dos semáforos, de acuerdo a la siguiente tabla.

Estado	Salida	Requerimiento
1	V1, R2	5 segundos
2	V1, R2	5 intermitentes de 200 ms, en V1
3	A1, R2	3 segundos
4	R1, V2	5 segundos
5	R1, V2	5 intermitentes de 200 ms, en V2
6	R1, A2	3 segundos

2.10.1. Pseudo código

Algoritmo 10: Control de dos semáforos

Configurar pines de semáforo 1 (V1, A1, R1);

Configurar pines de semáforo 2 (V2, A2, R2);

Mientras verdadero hacer;

 Encender V1 y R2;

 Esperar 5 s;

 Repetir 5 veces;

 Apagar V1;

 Esperar 200 ms;

 Encender V1;

 Esperar 200 ms;

 Apagar V1;

 Encender A1 y R2;

 Esperar 3 s;

 Encender R1 y V2;

 Esperar 5 s;

 Repetir 5 veces;

 Apagar V2;

 Esperar 200 ms;

 Encender V2;

 Esperar 200 ms;

 Apagar V2;

 Encender A2 y R1;

 Esperar 3 s;

2.10.2. Desarrollo

```
1 import machine
2 import utime
3
4 V1 = machine.Pin(0, machine.Pin.OUT)
5 A1 = machine.Pin(1, machine.Pin.OUT)
6 R1 = machine.Pin(2, machine.Pin.OUT)
7
8 V2 = machine.Pin(7, machine.Pin.OUT)
9 A2 = machine.Pin(6, machine.Pin.OUT)
10 R2 = machine.Pin(5, machine.Pin.OUT)
11
12 while True:
13     R1.value(0)
14     A2.value(0)
15     #E1: V1 y R2 prendidos 5 seg
16     V1.value(1)
17     R2.value(1)
18     utime.sleep_ms(5000)
19
20     #E2: V1 5 intermitentes 200ms, R2 prendido
21     for i in range(5):
22         V1.value(0)
23         utime.sleep_ms(200)
24         V1.value(1)
25         utime.sleep_ms(200)
26
27     #E3: A1 y R2 prendido 3 seg
28     V1.value(0)
29     A1.value(1)
30     utime.sleep_ms(3000)
31
32     #E4: R1 y V2 prendidos 5 seg
33     A1.value(0)
34     R1.value(1)
35     R2.value(0)
36     V2.value(1)
37     utime.sleep_ms(5000)
38
39     #E5: V2 5 intermitentes 200ms, R1 prendido
40     for i in range(5):
41         V2.value(0)
42         utime.sleep_ms(200)
43         V2.value(1)
44         utime.sleep_ms(200)
45
46     #E6: R1 y A2 prendidos 3 seg
47     V2.value(0)
48     A2.value(1)
49     utime.sleep_ms(3000)
```

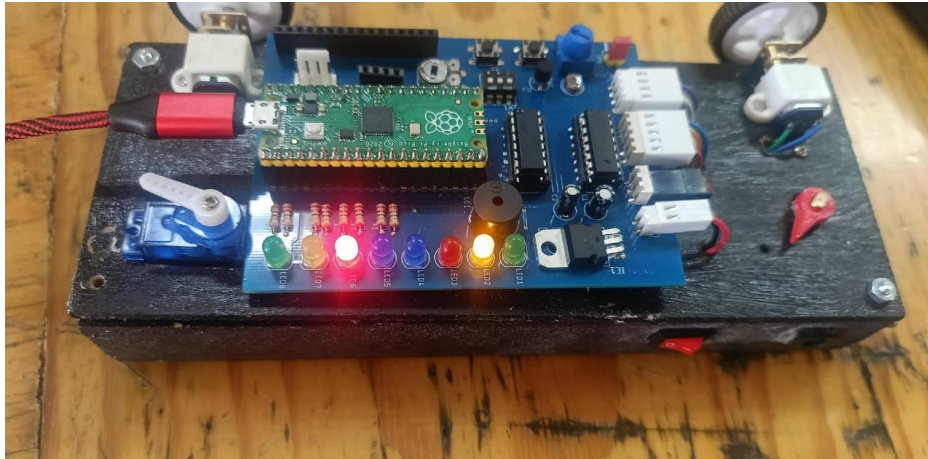


Figura 10: Tarjeta programada con el Ejercicio 10

2.10.3. Análisis de Resultados

Al ejecutar el programa se observa el funcionamiento de dos semáforos que operan de manera alternada simulando un cruce vehicular. Inicialmente el semáforo 1 permanece en verde mientras el semáforo 2 se encuentra en rojo durante 5 segundos. Posteriormente el LED verde del semáforo 1 parpadea indicando el cambio próximo a amarillo, el cual permanece encendido durante 3 segundos antes de pasar a rojo. En ese momento el semáforo 2 cambia a verde y se repite el mismo proceso de parpadeo y cambio a amarillo. Este comportamiento demuestra que la lógica de control implementada en el código permite coordinar correctamente las fases de ambos semáforos.

3. Conclusiones

El desarrollo de esta práctica se llevó a cabo gracias al tutorial de Raspberry Pi de la facultad [1] y a las clases del profesor Moisés [2].

3.1. Hernandez Acosta Mauricio Gabriel

A través de esta práctica pude familiarizarme de primera mano con la arquitectura y capacidades de la Raspberry Pi Pico, en especial con el microcontrolador RP2040. Trabajar con el IDE Thonny y MicroPython resultó ser una combinación muy amigable y eficiente, ya que simplifica notablemente la sintaxis sin perder el control directo sobre el hardware. Al desarrollar los algoritmos para los distintos ejercicios —desde el parpadeo básico de un LED hasta el manejo coordinado de múltiples salidas— me quedó claro cómo las instrucciones de código se traducen en señales lógicas reales en los pines GPIO. Esta experiencia me ayudó a comprender mejor la importancia de una correcta gestión de los retardos y cómo estructurar el código para que los periféricos respondan exactamente como se espera en un sistema embebido.

3.2. Perez Olvera Alexis Abraham

Cumplir con los objetivos de la práctica me permitió ver el potencial que tiene el RP2040 cuando se combina con la agilidad de MicroPython. Lo más interesante fue observar la evolución gradual de los programas: comenzamos con encendidos básicos y terminamos implementando secuencias más elaboradas, como el contador binario de 8 bits y la temporización cruzada de los dos semáforos. Esto demostró que el diseño algorítmico previo es fundamental antes de pasar a la implementación física, pues un pequeño desfase en los tiempos o un mal manejo de los índices en los pines GPIO altera por completo el comportamiento del circuito. En definitiva, la práctica me brindó bases sólidas para entender la manipulación de terminales digitales y la lógica secuencial en microcontroladores.

3.3. Sierra Garcia Mariana

El desarrollo de esta práctica fue fundamental para aterrizar los conceptos teóricos sobre cómo un microcontrolador interactúa con su entorno exterior. Mediante el uso de Thonny y MicroPython, logramos explorar a fondo las terminales GPIO del RP2040 y comprobar cómo una serie de ciclos y estructuras de control bien planteadas permiten gobernar el encendido simultáneo, progresivo y alternado de varios actuadores luminosos. Me pareció especialmente formativo el ejercicio de los semáforos y las secuencias de 8 bits, ya que representan escenarios de control y automatización cercanos a la realidad. Esta práctica reforzó mi confianza para programar sobre hardware embebido y me deja una visión clara de cómo diseñar algoritmos eficientes para el control de puertos de entrada y salida.

Referencias

- [1] R. Anaya García et al., *Manual de Aplicaciones Raspberry Pi*, Manual de recursos y aplicaciones para plataforma Raspberry Pi, Universidad Nacional Autónoma de México, Facultad de Ingeniería, Laboratorio de Microcomputadoras, Ciudad Universitaria Coyoacán, México, 2026. dirección: <https://odin.fi-b.unam.mx/micros/docs/Tutoriales%20Raspberry.pdf>
- [2] M. Meléndez Reyes, *Notas de clase: Práctica 01 - Introducción a las arquitecturas ARM empleando Raspberry Pi*, Clase presencial en laboratorio de microprocesadores, Facultad de Ingeniería, UNAM, Material proporcionado por el profesor durante la sesión de laboratorio, feb. de 2026.